Experiment 17 — Neural-network training: 123× behind, on one missing function

Mathilda — execution-speed experiments

Contents

Experiment 17 — Neural-network training: 123× behind, on one missing function

Date: 2026-07-31 · Code: `src/piecewise.c`, `src/ndkernels.c`, `src/info.c` · Result: MLP training 44.69 s → 3.90 s (11.5×); inference 344 → 17.9 ms, now 1.09× ahead of NumPy

Common method in `README.md`.

Hypothesis

The third sweep's logistic-regression row is the nearest existing benchmark and it has one weight matrix. Real training has a chain of them, and the backward pass is a chain of transposed products:

`dW2 = a1ᵀ · d2` `dh = d2 · W2ᵀ` `dW1 = Xᵀ · d1`

Every one of those is a `Transpose` feeding a `Dot`, and the fourth sweep left `Transpose` as its single largest named application gap (plan item 9.1: NumPy's `.T` is a view, Mathilda's is a copy). A two-layer MLP is the smallest kernel that exercises the whole chain.

Bias vectors are folded in as a constant column of `X` rather than added separately. That is what a performance-minded implementation does in all three languages, and it keeps the three columns identical — Wolfram's `Plus` threads the outer level, so `matrix + rowVector` is not a broadcast and the honest spelling would otherwise have to differ per system.

The kernels

id	kernel	what it stresses
`mlptrain`	785-128-10, batch 1024, 100 SGD steps	GEMM chain, transposes, softmax, ReLU, loop-carried weights
`mlpinfer`	8192 × 785 forward pass	one large GEMM, ReLU, softmax

The check is a deterministic 256 × 17 → 8 → 4 instance trained for 20 steps, with the cross-entropy loss compared across all three systems; the timed run uses random data, which no two systems can be made to share.

What the first run found

	Mathilda	Mathematica	NumPy
MLP training, 100 steps	44.69 s	364 ms	356 ms
MLP inference, 8192 × 785	344 ms	10.8 ms	19.5 ms

123× behind Mathematica and 126× behind NumPy. By a distance the worst row in the sweep, and the two comparison columns agree with each other to 2%, which is itself informative: both of them are just calling BLAS and a vectorised `maximum`, so the gap is entirely Mathilda's.

The cause: the rectified linear unit could not be spelled

`Ramp` did not exist, and `Clip[x, {0., Infinity}]` returned unevaluated because the infinite bound failed to parse. The only working spelling of `max(x, 0)` over an array was `x UnitStep[x]` — two passes and a mixed `Real`/`Integer` product.

That is documented in full in `OPTION_PRICING.md`, which hit the same wall from the early-exercise side. `Ramp` is now a builtin with the Mathematica attribute set and a threaded buffer kernel:

10^6 float64	before	after
`Ramp[v]`	— (did not exist)	1.41 ms
`v UnitStep[v]` — the only prior spelling	14.5 ms	14.5 ms

Results

Benchmark	before	after	Mathematica 14.0	NumPy 2.4.4	
MLP training, 785-128-10, 100 steps	44.69 s	3.90 s	364 ms	356 ms	11.5×
MLP inference, 8192 × 785	344 ms	17.9 ms	10.8 ms	19.5 ms	1.09× ahead of NumPy

Inference — one GEMM, a ReLU and a softmax — is now at NumPy parity. Training is 11.5× better than it was and still 10.7× behind both comparison systems, and this is the row where the remaining gap is best understood in the whole suite.

Where the remaining 10× is, measured

Per training step, timed directly:

	per step
`Transpose[X]`, 1024 × 785	2.05 ms
`Transpose[X] . dz1` (including the transpose)	3.19 ms
`da1 UnitStep[z1]` — the ReLU derivative mask	1.37 ms

	per step
`X . W1` (1.03 × 10⁸ MACs, `dgemm`)	0.99 ms
`Transpose[a1] . d2`	0.48 ms
`Exp[z2 - Map[Max, z2]]`	0.37 ms
`Ramp[z1]`	0.20 ms
`W1 - lr g1`	0.46 ms

Two lines account for most of it, and neither is arithmetic.

`Transpose[X]` is loop-invariant and re-evaluated 100 times. `X` does not change; its transpose is recomputed every step at 2.05 ms, which is twice the cost of the forward *dgemm* it feeds. NumPy's `.T` is a view and costs nothing. This is plan item 9.1 and it is a design change — a strided/transposed `NDArray` view that every consumer must honour — not a fast path. It is the same missing feature as the sliding-window view in experiment 14.

The mask multiply **`da1 UnitStep[z1]`** costs 1.37 ms for 131072 elements, or 10.5 ns per element, where a float64 multiply of the same array is about 0.4 ns/element. `UnitStep` correctly narrows to an exact `int64` buffer, and the mixed `float64 × int64` elementwise product then goes through the generic accessor pair rather than a widening loop. That is a contained fix — one dtype pair in `ndarray_elementwise` — and it is worth about 25× on this line.

What is still open

1. A transposed view (plan 9.1) — worth ~2 ms/step here, and the same mechanism closes experiment 14's sliding window.
2. Mixed **`float64 × int64`** elementwise — worth ~1.3 ms/step here, and it appears anywhere a comparison mask multiplies real data, which is most branch-free array code.
3. Unfused composition (plan 9.2) — the softmax alone is four passes over the logits.

Those three together are most of the remaining 10×, and none of them is specific to neural networks.

Why Mathilda is not the fastest here, and what it would take

row	Mathilda	best other	gap	cause
MLP training, 100 steps	3.90 s	356 ms (NumPy)	10.9×	a ragged weight tuple materialised every step, then a copyi
MLP inference	17.9 ms	10.8 ms (WL)	1.67×	one GEMM plus a softmax; unfused

Splitting one training step, against NumPy on the same data (`neural_network_training.m` and `.py` print this):

per step	Mathilda	NumPy	note
`nnX . W1` — the forward GEMM	0.848 ms	0.968 ms	Mathilda 1.14× ahead
`Transpose[nnX]`	1.756 ms	0.000 ms	NumPy's `.T` is a view
`Transpose[nnX] . dz1`	2.915 ms	0.967 ms	the GEMM is at parity; the copy is the gap
`da1 UnitStep[z1]` — the ReLU mask	1.156 ms	0.189 ms	mixed `float64 × int64`
ReLU, softmax shift	0.61 ms	0.27 ms	unfused passes

The GEMMs are already at parity or ahead — both systems call the same Accelerate `dgemm`. Nothing on this row is about matrix multiplication.

But the split sums to about 8 ms and a step costs 39 ms. The missing 31 ms is the finding:

`Nest[mlpstep, {W1, W2}, 100]` carries the weights as `{W1, W2}` — 785×128 and 128×10, different shapes. A list of packed rows is absorbed into one buffer only if the rows agree in shape and dtype; a ragged pair declines, and the no-nesting invariant then materialises both weight matrices, every step.

This is experiment 13's mixed-dtype tuple in its third form, and it is worse, because a neural network's weights are always ragged. Measured directly: building `{W1, W2}` costs 9.9 ms where building a same-shape pair costs 0.70 ms, and the caller then pays 15× on its next use of `w[[1]]`.

The road to fastest, with the payoff of each step measured

The three steps below were each run as a variant of the real kernel, verified to produce bit-identical weights (`Total[Abs[uW1 - vW1], 2]` is exactly `0.`):

variant	100 steps	vs the next
as written — ragged tuple through `Nest`	4.02 s	
A. weights held as two separate values	0.99 s	4.07×
B. A, plus the loop-invariant `Transpose` hoisted	0.72 s	1.36×

1. A heterogeneous packed tuple — a container that can hold n buffers of independent shape and dtype without materialising them. This is the single highest-value open item in the whole suite: 4.07× here, and it also closes experiment 13's 120× return-statement finding, because ragged and mixed-dtype are the same defect. `List` itself cannot take this role without breaking the transparency gate's O(argc) scan, so it wants a node type of its own.
2. A transposed view (plan 9.1), or the cheaper 80%: have `Dot` recognise `Transpose[a] . b` and pass `CblasTrans` to BLAS, which costs nothing. 1.36× on top of (1).
3. Mixed **`float64 × int64`** elementwise through a widening loop rather than the generic accessor pair. Worth ~1 ms of the remaining ~7 ms/step, and it appears anywhere a comparison mask multiplies real data.
4. Interpreter-level fusion (plan 9.2) for the softmax and the update — four passes over the logits where one would do.

Steps 1 and 2 take the row to 0.72 s against NumPy's 0.356 s; 3 and 4 are what would close the last 2×. Since the GEMMs are already ahead, being fastest overall is a matter of not paying for anything else.

Verification

- The loss check on a deterministic instance agrees to full printed precision across all three systems, before and after.
- Ramp's differential and reference-value tests are in `test_fifth_sweep_fast_paths`; see OPTION_PRICING.md.